Computational Methods in Economics - Problem Set 3

Luis Felipe Lins

March 2026

Introduction

In this problem set, we explore core numerical methods in computational economics: optimization, numerical integration, and numerical differentiation. The repository containing the codes used in this problem set's solution can be found on GitHub.

1 Question 1

We want to minimize $f(x) = x\sin(5x)$ over the interval $[0, 10]$, using three different optimization methods with absolute and relative tolerances of 10^{-4}.

Item (a)

I chose the following three methods: Nelder-Mead (a derivative-free simplex method), L-BFGS-B (a quasi-Newton method that supports bounds), and Differential Evolution (a global optimizer from the family of evolutionary algorithms). All three are called through scipy.optimize, with starting point $x_0 = 0$ for the local methods. The results are:

Method	x^*	$f(x^*)$
Nelder-Mead	0.0000	0.0000
L-BFGS-B	0.0000	0.0000
Differential Evolution	9.7430	-9.7410

Table 1: Optimization results for $f(x) = x\sin(5x)$ on $[0, 10]$.

Both local methods converge to $x^* = 0$, which is a local minimum (and the boundary of the interval). Differential Evolution, being a global optimizer, successfully finds the global minimum at $x^* \approx 9.743$, with $f(x^*) \approx -9.741$.

Figure 1 illustrates why local methods struggle: the function has many local minima due to the oscillatory nature of $\sin(5x)$, and the amplitude of the oscillations grows with x. Starting at $x_0 = 0$, the local methods have

no reason to "jump" past the first few oscillations and get stuck immediately.

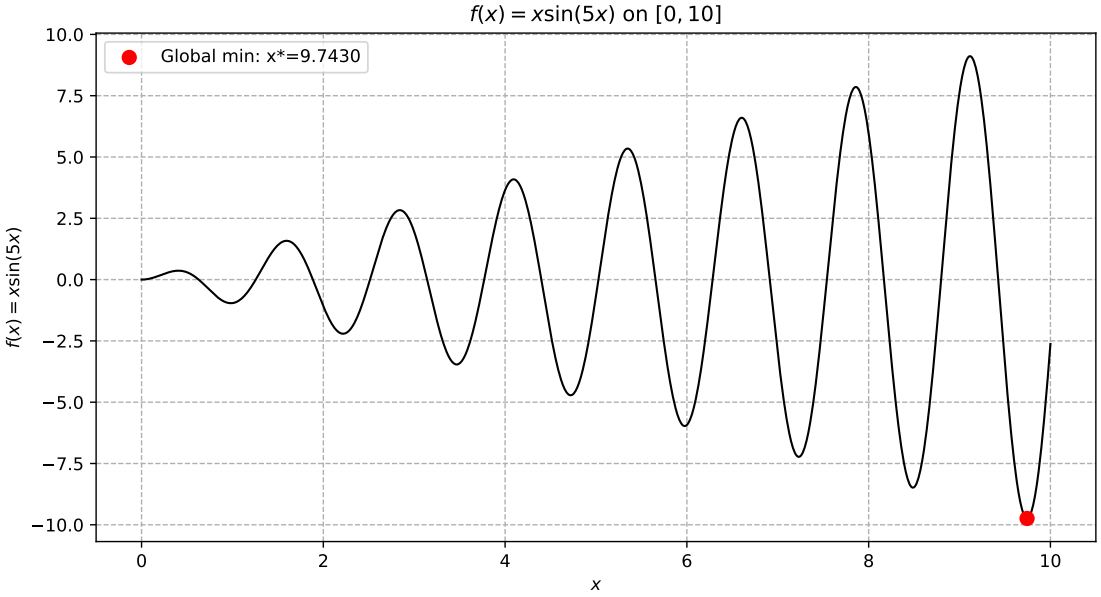


Figure 1: $f(x) = x\sin(5x)$ on $[0, 10]$ with the global minimum marked in red.

Item (b)

To investigate how the solution depends on the starting point and the algorithm, I ran both Nelder-Mead and L-BFGS-B from 50 evenly spaced starting points across $[0, 10]$, and plotted the resulting $f(x^*)$ for each starting point.

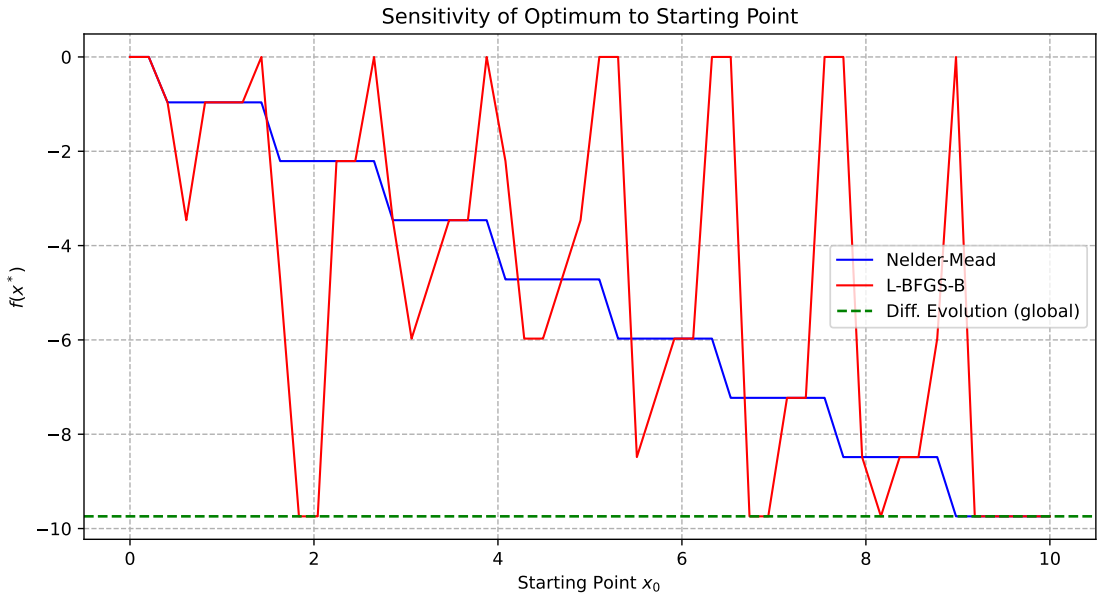


Figure 2: Optimum $f(x^*)$ as a function of the starting point x_0 for each method

The results in Figure 2 make the point clearly: both local methods are highly sensitive to the starting point,

converging to different local minima depending on where they start. Neither method consistently finds the global optimum. L-BFGS-B and Nelder-Mead follow broadly similar patterns, though they occasionally converge to different local minima for the same x_0, reflecting differences in how the two algorithms navigate the landscape (gradient-based steps vs. simplex reflections). Differential Evolution, by contrast, does not require a starting point and explores the entire domain, reliably finding the global minimum.

2 Question 2

We are given the function:

$$f(x_1, x_2; \theta) = \theta_1 x_1 + \theta_2 \exp(-x_2^2) + \theta_3 \log(1 + |x_2|) + \theta_4 x_1^{x_2}$$

and four observed values $y_1 = 43.614$, $y_2 = 563.694$, $y_3 = 43.230$, $y_4 = 23.130$ at known input pairs. The goal is to recover the unknown parameter vector $\theta_0 \in \mathbb{R}^4$ by minimizing the sum of squared residuals:

$$g(\theta) = \sum_{k=1}^{4} \Big(f(x_1^{(k)}, x_2^{(k)}; \theta) - y_k \Big)^2$$

Item (b)

As a sanity check, evaluating g at $\theta_1 = (0, 0, 0, 0)$ gives $g(\theta_1) = 322056.94$. This is expected: with all parameters set to zero, the model predicts zero for all observations, so g simply returns the sum of squared observed values.

Items (c-e)

I minimize $g(\theta)$ using Nelder-Mead with starting point $\theta_0 = (1, 1, 1, 1)$ and tolerances xatol = fatol = 10^{-8}. At each iteration, a CallbackRecorder object stores the current parameter vector θ_i and the value $g(\theta_i)$.

The optimizer converges after 412 iterations to:

$$\hat{\theta} = (1.80,\ 2.40,\ 10.00,\ 34.00)$$

with $g(\hat{\theta}) \approx 0$, indicating an essentially perfect fit to the four observations. This makes sense: we have four equations and four unknowns, so the system is exactly identified and we should expect an exact solution to exist.

Figure 3 shows the convergence of $g(\theta_i)$ across iterations. The objective function drops rapidly in the first few iterations and then slowly approaches zero as the algorithm fine-tunes the parameters. Figures 4a to 4d show the path of each individual parameter across iterations.

3 Question 3

Suppose $X, Y \sim N(0, 1)$ and $u = \max\{X, Y\}$. We want to compute $\mathbb{E}[u]$ using two different numerical integration methods. The analytical solution is known to be $\mathbb{E}[\max\{X, Y\}] = 1/\sqrt{\pi} \approx 0.5642$.

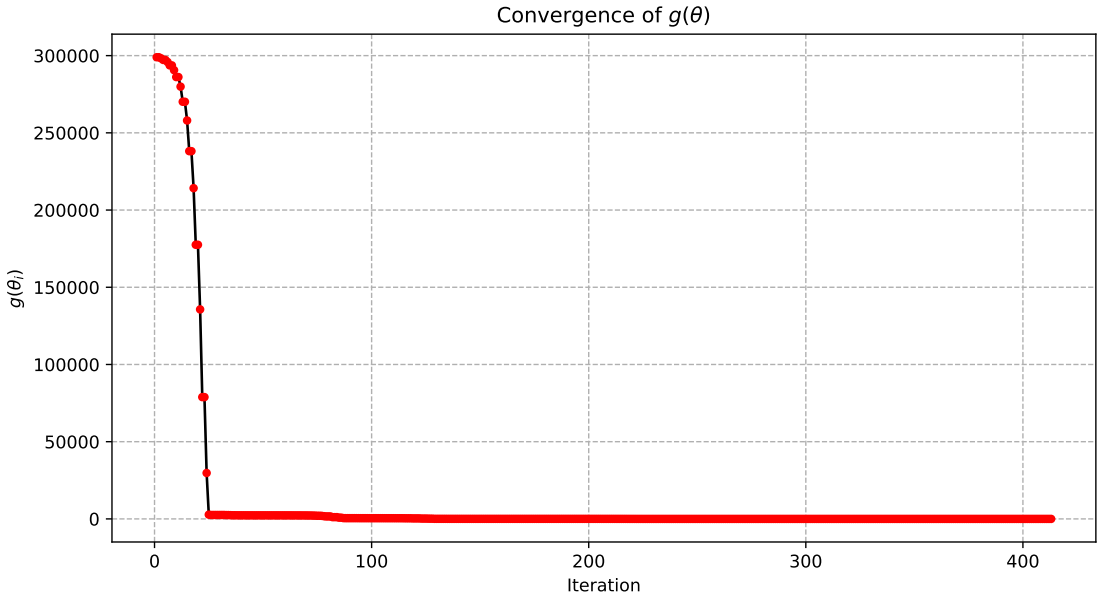
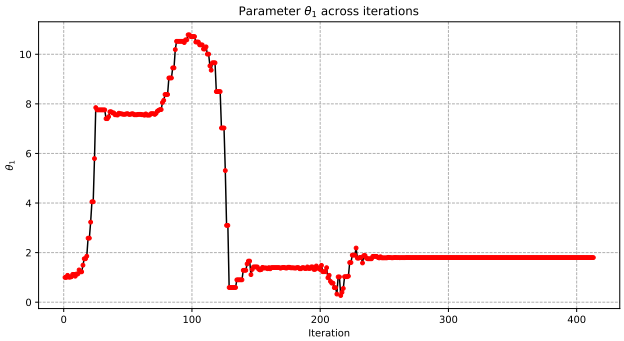
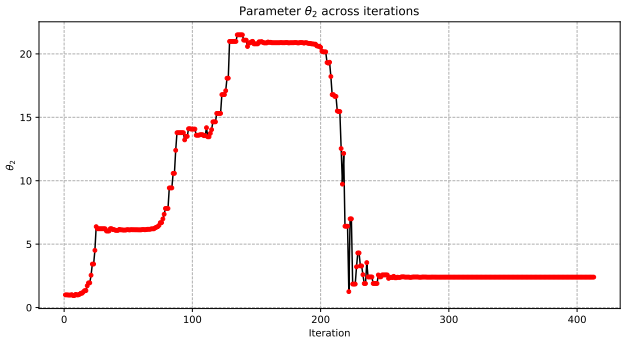


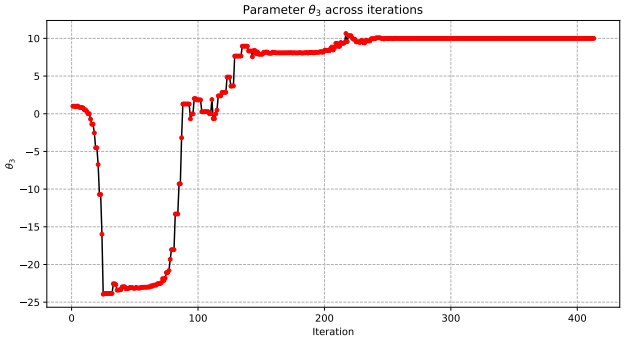
Figure 3: Convergence of $g(\theta_i)$ across Nelder-Mead iterations.



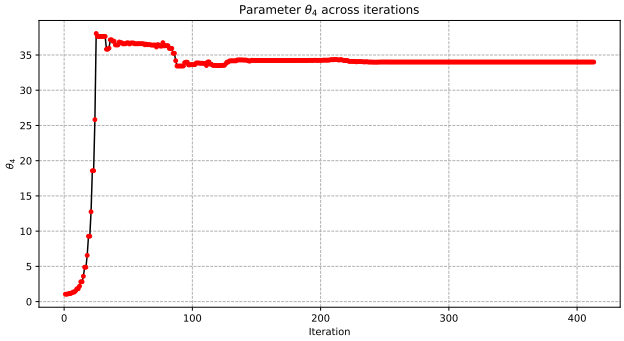
(a) θ_1



(b) θ_2



(c) θ_3



(d) θ_4

Figure 4: Parameter paths $\theta_1, \ldots, \theta_4$ across Nelder-Mead iterations.

Item (a)

The key idea is that the standard normal density can be connected to the Gauss-Hermite weight function. With the substitution $x = \sqrt{2}\,t$, we get:

$$\int_{-\infty}^{\infty} h(x) \frac{1}{\sqrt{2\pi}} e^{-x^2/2}\, dx = \frac{1}{\sqrt{\pi}} \int_{-\infty}^{\infty} h(\sqrt{2}\,t)\, e^{-t^2}\, dt$$

which is exactly the form that Gauss-Hermite quadrature approximates. For a two-dimensional integral (since we need both X and Y), we use the product rule: the 2D integral becomes a double sum over the tensor product of the 1D nodes and weights.

Nodes (n)	$\hat{\mathbb{E}}[\max\{X, Y\}]$	|error|
5	0.5184	4.58×10^{-2}
10	0.5413	2.29×10^{-2}
20	0.5527	1.15×10^{-2}
50	0.5596	4.63×10^{-3}
100	0.5619	2.32×10^{-3}

Table 2: Gauss-Hermite quadrature estimates of $\mathbb{E}[\max\{X, Y\}]$ by number of nodes per dimension.

The convergence is notably slow: even with 100 nodes per dimension (10,000 total evaluation points), the error is still around 2.3×10^{-3}. This is because $\max\{x, y\}$ has a kink along the line $x = y$, which breaks the smoothness that Gauss-Hermite quadrature relies on for fast polynomial convergence.

Item (b)

The Monte Carlo approach is straightforward: draw n pairs (X_i, Y_i) from $N(0, 1)$ and compute the sample average of $\max\{X_i, Y_i\}$.

Draws (n)	$\hat{\mathbb{E}}[\max\{X, Y\}]$	|error|
1,000	0.5139	5.03×10^{-2}
10,000	0.5749	1.07×10^{-2}
100,000	0.5653	1.07×10^{-3}
1,000,000	0.5643	8.45×10^{-5}

Table 3: Monte Carlo estimates of $\mathbb{E}[\max\{X, Y\}]$ by number of draws.

With 1 million draws, the error is already below 10^{-4}, substantially beating the Gauss-Hermite estimate at 100 nodes. This illustrates the trade-off that deterministic quadrature methods converge faster for smooth integrands, but Monte Carlo is more robust to non-smooth functions and, importantly, does not suffer from the curse of dimensionality, making it the method of choice for higher-dimensional integration problems common in economics.

4 Question 4

We compute the integrals of three functions over $[0, 1]$ using the trapezoid rule with $n = 3, 5, 10, 15, 20$ nodes.

	$n = 3$	$n = 5$	$n = 10$	$n = 15$	$n = 20$
$\int_0^1 x\,dx = 0.5$	0.5000	0.5000	0.5000	0.5000	0.5000
$\int_0^1 x\sin(x)\,dx \approx 0.3012$	0.3302	0.3084	0.3026	0.3018	0.3015
$\int_0^1 \sqrt{1-x^2}\,dx = \pi/4 \approx 0.7854$	0.6830	0.7489	0.7745	0.7798	0.7819

Table 4: Trapezoid rule approximations by number of nodes.

A few observations. First, the integral $\int_0^1 x\,dx$ is computed exactly for all n since the trapezoid rule is exact for linear functions (the "trapezoids" fit the function perfectly). Second, $\int_0^1 x\sin(x)\,dx$ converges quickly: by $n = 20$ the error is on the order of 10^{-4}. Third, $\int_0^1 \sqrt{1-x^2}\,dx$ converges more slowly. This happens because $\sqrt{1-x^2}$ has an infinite derivative at $x = 1$ (vertical tangent), which degrades the accuracy of the trapezoid rule near the boundary.

5 Question 5

We compute the derivative of three functions at specified points using 2-point and 4-point centered differences, for step sizes $h = 0.001, 0.005, 0.01, 0.05$.

The 2-point centred difference formula is $f'(x) \approx \frac{f(x+h)-f(x-h)}{2h}$.

The 4-point formula is $f'(x) \approx \frac{-f(x+2h)+8f(x+h)-8f(x-h)+f(x-2h)}{12h}$.

Item (a): $f(x) = x^2$ at $x = 5$

The analytical derivative is $f'(5) = 10$. Both methods produce errors at the level of machine precision, regardless of h:

h	|error| (2-pt)	|error| (4-pt)
0.001	3.34×10^{-12}	4.52×10^{-12}
0.005	2.13×10^{-13}	2.13×10^{-13}
0.010	2.13×10^{-13}	2.72×10^{-13}
0.050	3.55×10^{-14}	2.49×10^{-14}

Table 5: Absolute errors for $f'(x) = 2x$ at $x = 5$.

This is because x^2 is a polynomial of degree 2, and the 2-point centered difference formula is exact for polynomials up to degree 2. The residual errors are a result of floating-point arithmetic.

Item (b): $f(x) = \log(x)$ at $x = 10$

The analytical derivative is $f'(10) = 0.1$. Here we can clearly see the difference in convergence rates between the two methods:

h	|error| (2-pt)	|error| (4-pt)
0.001	3.33×10^{-10}	1.59×10^{-13}
0.005	8.33×10^{-9}	4.06×10^{-14}
0.010	3.33×10^{-8}	7.39×10^{-14}
0.050	8.33×10^{-7}	5.00×10^{-11}

Table 6: Absolute errors for $f'(x) = 1/x$ at $x = 10$.

The 2-point errors decrease as we reduce h. The 4-point errors are dramatically smaller, mostly near machine precision for all values of h, albeit lower values produce lower errors. Starting at a very low h (0.001), the error starts to increase, probably because of machine precision.

Item (c): $f(x) = x\sin(x)$ at $x = 1$

The analytical derivative is $f'(1) = \sin(1) + \cos(1) \approx 1.3818$. The pattern is similar to item (b):

h	|error| (2-pt)	|error| (4-pt)
0.001	5.11×10^{-7}	3.28×10^{-13}
0.005	1.28×10^{-5}	9.89×10^{-11}
0.010	5.11×10^{-5}	1.58×10^{-9}
0.050	1.28×10^{-3}	9.89×10^{-7}

Table 7: Absolute errors for $f'(x) = \sin(x) + x\cos(x)$ at $x = 1$.

The 4-point method achieves errors several orders of magnitude smaller than the 2-point method for the same step size.

Item (d): Comparison with the analytical solution

Across all three functions, the results confirm the theoretical convergence rates. In practice, this means that with a moderate step size like $h = 0.01$, the 4-point method already achieves errors near machine precision for smooth functions, while the 2-point method still has errors of order 10^{-8} to 10^{-5}.

The one exception is $f(x) = x^2$, where both methods are essentially exact because the higher-order derivatives that drive the truncation error are zero. This serves as a useful sanity check for the implementation.